

José Adriel Marcos
Raúl Cristian Dumitru Tanase





ÍNDICE

ÍNDICE	2
Arquitectura de Software	3
Stack tecnológico	4
Frontend:	4
Backend:	4
Base de datos:	4
DevOps:	4
Estructura de carpetas	5
Backend:	5
Frontend:	5
Sistema de auth	6
Registro:	6
Inicio de sesión:	6
Modelo de datos	7
user:	7
vehicles:	7
emulatedData:	7
realData:	8
alerts:	8
Relaciones:	8
API REST	8
Sistema de alertas	10
Componentes del frontend	10
Estructura general:	10
Páginas principales:	11
Componentes compartidos (Misc):	11
Filtros y búsqueda	11



Arquitectura de Software

La arquitectura seleccionada para la aplicación web es la Hexagonal DDD (Domain Driven Design). Es correcto reconocer que para este proyecto la arquitectura genera bastante overengineering dado el leve tráfico que soportará la aplicación. De todas formas, lo que también es correcto, es realizar un planteamiento a futuro sobre las expectativas de un proyecto, si este proyecto continuará y creciera de manera exponencial los futuros desarrolladores lo tendrían más fácil para programar sobre el mismo.

Esta arquitectura es reconocida por su gran organización separando la lógica de negocio del resto del código. La aplicación funciona como un hexágono con puertos donde entra y sale información con indiferencia del origen. No importa si los datos provienen de API REST, CLI, test, etc...

Podemos separar el funcionamiento en tres grandes apartados; Domain, application e infraestructure.

- Domain: Es el núcleo de la aplicación, en este punto se encuentran las entidades (user, vehicle, etc) que representan los objetos del negocio. Dichos objetos aplican reglas y validaciones propias. A su vez, podemos encontrar servicios (realImpactAnalysisService) que contienen lógica de negocio y por último interfaces de repositorio (IRealDataRepository) las cuales definen qué operaciones se precisan sobre los datos pero sin explicar cómo se hacen.
- Application: En este caso encontramos los casos de uso (useCases). Cada archivo es una acción concreta que el sistema puede hacer (createRealData, loginUser). Resumiendo, orquestan la lógica; reciben datos, llaman al dominio, llaman al repositorio y devuelven el resultado.
- Infraestructure: En primer lugar contamos con los adapters IN, son archivos que definen los controllers y rutas HTTP haciendo que el useCase lo pueda entender. En segundo lugar contamos con los adapters OUT, definen como accede la aplicación web a datos externos. Es decir, la base de datos, en este caso se ha utilizado Sequelize y MySQL.

Para evitar el acoplamiento fuerte entre capas existe el archivo "container". Se encarga básicamente de realizar inyecciones de dependencias instanciando repositorios concretos y pasandoselos a los useCases de manera que nunca dependan de una implementación específica. Si el día de mañana Sequelize es cambiado por su equivalente de MongoDB solo habrá que cambiar los repositorios y el container, sin tocar lógica de negocio.



Stack tecnológico

Frontend:

- React 19: Librería de UI que se basa en componentes reutilizables.
- Vite: Servidor de desarrollo.
- React Router: Gestión de rutas/navegación.
- Axios: Cliente HTTP que se comunica con el backend.
- CSS puro: Sin uso de frameworks como Tailwind.

Backend:

- Node.js: Entorno de ejecución de JS en un servidor.
- Express 5: framework de HTTP para crear API REST.
- Sequelize-CLI: ORM para interactuar con la DB sin sentencias SQL.
- JWT: Generación de tokens para autenticación.
- argon2: Hashing de contraseñas.
- cookie-parser: Handling de cookies httpOnly.
- express-rate-limit: Limitar peticiones http por IP.
- express-validator: Validación de inputs.

Base de datos:

- MySQL: Gestor de bases de datos relacionales.

DevOps:

- PM2: Gestor de procesos en producción de node.js.
- Nginx: Servidor web usado para reverse proxy.
- AWS Lightsail: Instancia cloud para el deploy.



Estructura de carpetas

Backend:

```
server/src/
├── application/
│   ├── useCases/      → Casos de uso organizados por entidad
│   │   ├── alerts/
│   │   ├── emulatedData/
│   │   ├── realData/
│   │   ├── user/
│   │   └── vehicle/
│   ├── domain/
│   │   ├── entities/  → Objetos de negocio (User, Vehicle, Alert...)
│   │   ├── repositories/ → Interfaces de repositorio (contratos)
│   │   └── services/   → Lógica de negocio pura (análisis de impacto)
│   └── infraestructure/
│       ├── adapters/
│       │   ├── in/http/    → Controllers y rutas HTTP
│       │   ├── validators/ → Validaciones de entrada (express-validator)
│       │   └── out/persistence/ → Repositorios Sequelize (implementaciones)
│       ├── config/         → Configuración de DB y container de dependencias
│       ├── db/
│       │   ├── migrations/ → Migraciones de base de datos
│       │   ├── seeders/    → Datos de prueba
│       │   └── services/   → Servicios de infraestructura
```

Frontend:

```
client/src/
├── assets/          → Imágenes, iconos, recursos estáticos
├── components/      → Componentes React organizados por página
│   ├── AboutUs/     → Hero, Mission, Team, Values
│   ├── CrashNotification/ → Notificación de colisión
│   ├── Dashboard/   → Panel principal
│   ├── Demo/        → Hero, Form, FAQ
│   ├── Documentation/ → Header, lista con visor PDF
│   ├── LandingPage/  → CTA, Hero, Partners, Platform
│   ├── Misc/        → Componentes compartidos (NavBar, Footer, LoginForm, LogsTable...)
│   └── TechnologyCards/ → Tarjetas de tecnología
├── context/         → Contextos de React (AuthContext, etc.)
├── layouts/          → Layouts (MainLayout, NoFooterLayout)
├── pages/           → Páginas que usan los componentes
├── router/          → Definición de rutas (React Router)
├── services/        → Llamadas HTTP al backend (apiClient)
└── utils/           → Utilidades compartidas
```



Sistema de auth

Registro:

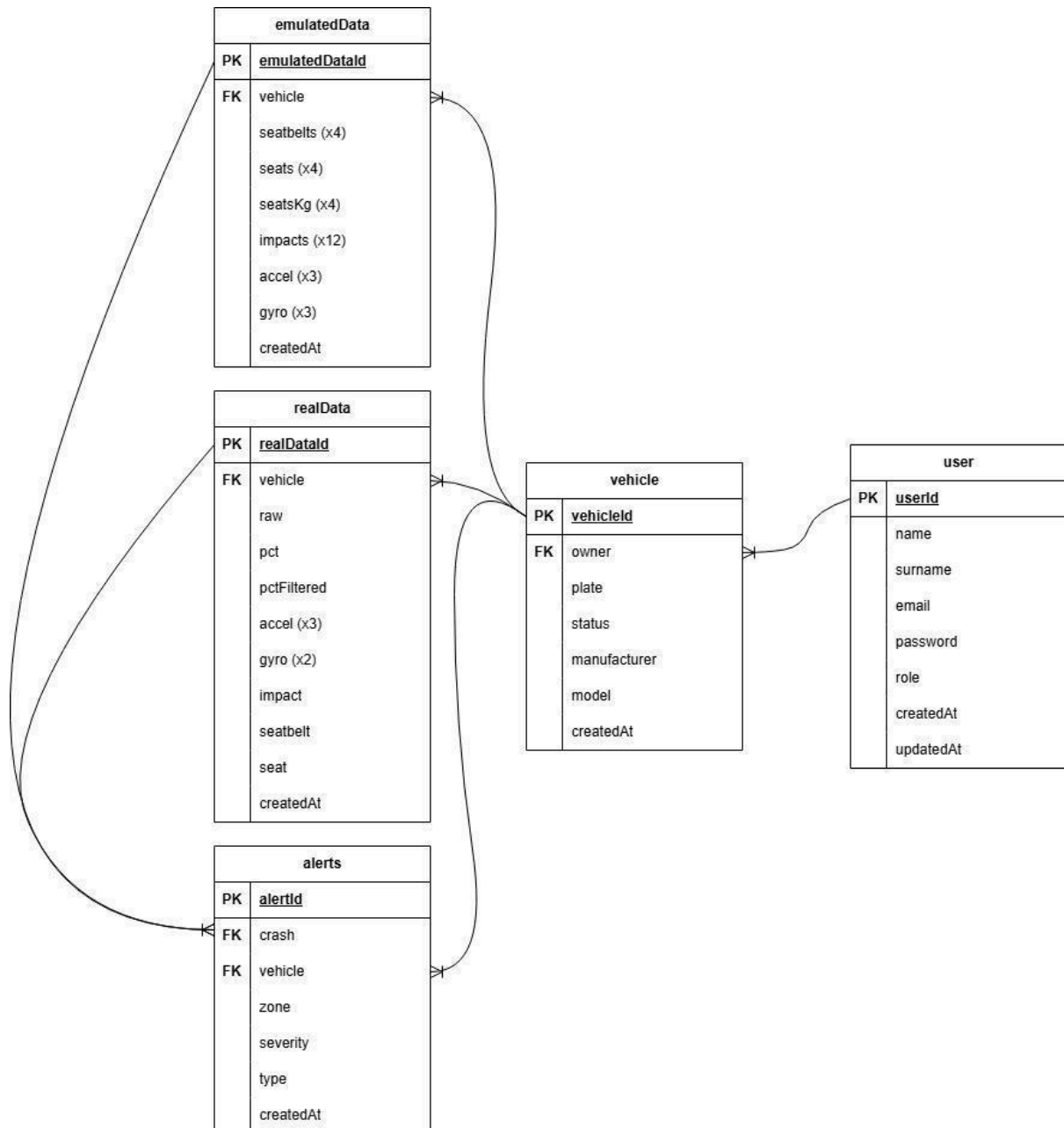
- El registro actualmente solo se puede realizar mediante postman dado que no existe sistema integrado en el frontend que lo realice, esto se ha tomado como una implementación a futuro para el rol de professional (+). De todas formas, existe un flujo en el sistema de registro:
 1. Se rellena el body en postman con todos los campos necesarios; name, surname, email y password.
 2. La contraseña se hashea con argon2 (nunca se guarda en texto plano).
 3. Se crea el usuario en la base de datos.

Inicio de sesión:

- En este caso el login si que tiene su correspondiente sistema integrado en el frontend mediante un componente albergando el formulario y la respectiva lógica del mismo y una page que alberga dicho componente. El flujo es el siguiente:
 1. El usuario envía su email y contraseña.
 2. Se busca el usuario en la base de datos.
 3. Se compara la contraseña con el hash guardado en la base de datos.
 4. Si es correcta se genera un JWT con los datos no sensibles del usuario.
 5. El token se guarda 24h dentro del navegador con una cookie httpOnly.
- Durante el desarrollo surge una pregunta importante ¿Por qué httpOnly y no localStorage? El resumen más simple es seguridad, una cookie httpOnly no es accesible desde JavaScript, por lo tanto no puede ser atacada con scripts maliciosos y robar el token como sí sucede con localStorage.
- Por tanto, el hecho de que la aplicación web posea un sistema de autenticación implica que este mismo sistema limitará muchas de las acciones que se pueden realizar. Esto se hace con un middleware de autenticación que dentro de los archivos se puede encontrar como "authMiddleware". El flujo para las peticiones autenticadas:
 1. El navegador envía la cookie automáticamente con cada petición.
 2. El middleware extrae el token de la cookie.
 3. Verifica el JWT con la clave secreta del .env.
 4. Si el token es válido, mete los datos del usuario en req.user y deja pasar la petición.
 5. Si no es válido, devuelve el status 401 (Unauthorized).



Modelo de datos



user:

- Almacena los usuarios registrados en el sistema. Un usuario puede ser client, professional o admin. Además, un usuario puede tener n vehículos asociados.

vehicles:

- Representa los vehículos monitorizados. Cada vehículo pertenece a un solo usuario. Es una tabla sin timestamps.

emulatedData:

- Datos generados por el simulador. Contiene diferentes sensores y cada registro pertenece a un solo vehículo.



realData:

- Datos reales generados desde los sensores de arduino. Cada registro pertenece a un solo vehículo.

alerts:

- Alertas generadas automáticamente cuando se detecta un impacto. El campo type indica si los datos por los cuales salta la alerta provienen de la tabla emulatedData o realData. El campo crash referencia el ID del registro correspondiente.

Relaciones:

- Un user tiene muchos vehicles (1:N)
- Un vehicle tiene muchos emulatedData, realData y alerts (1:N)
- Una alert referencia un registro de emulatedData o realData segun su type (polimórfica)

API REST

<u>User (/api/user)</u>			
<u>Método</u>	<u>Ruta</u>	<u>Auth</u>	<u>Descripción</u>
POST	/register	Sí	Registro de usuario
POST	/login	No (rate-limit)	Inicio de sesión
POST	/logout	Sí	Cerrar sesión
GET	/me	Sí	Obtener usuario actual
GET	/findAllByRole	Sí	Listar usuarios por rol

<u>Vehicles (/api/vehicles)</u>			
<u>Método</u>	<u>Ruta</u>	<u>Auth</u>	<u>Descripción</u>
POST	/create	Sí + validación	Crear vehículo
GET	/getAll	Sí	Listar todos los vehículos
DELETE	/:id	Sí	Eliminar vehículo



<u>Emulated Data (/api/emulatedData)</u>			
<u>Método</u>	<u>Ruta</u>	<u>Auth</u>	<u>Descripción</u>
POST	/create	Sí + validación	Crear dato emulado
GET	/search	Sí	Buscar por filtros
GET	/countHourly	Sí	Conteo de última hora

<u>Real Data (/api/realData)</u>			
<u>Método</u>	<u>Ruta</u>	<u>Auth</u>	<u>Descripción</u>
POST	/create	API Key	Crear dato real
GET	/search	Sí	Buscar por filtros
GET	/countHourly	Sí	Conteo de última hora

<u>Alerts (/api/alerts)</u>			
<u>Método</u>	<u>Ruta</u>	<u>Auth</u>	<u>Descripción</u>
GET	/getAll	Sí	Listar todas las alertas
GET	/getLatest	Sí	Obtener alerta más reciente

Todas las rutas van bajo /api/ dado que sigue el patrón REST. Casi todos las rutas poseen el authMiddleware dado que son rutas llamadas por un usuario. Sin embargo, /api/realData/create posee apiKeyMiddleware para que el equipo de arduino pueda enviar datos autenticados desde una key y no desde un usuario.

Por otra parte, otro de los middleware usados es el rate limiter en /api/user/login. Esto se hace para limitar por IP las peticiones de inicio de sesión a 10. Esto es útil dado que protege contra ataques de fuerza bruta.

Otras rutas tienen sus propios middleware de validación de la información que entra en el req.body. Se asegura de que un email por ejemplo tenga formato de email. De no haber usado ORM, también sería útil para proteger contra inyecciones SQL maliciosas, en este caso no es así dado que Sequelize ya lo integra al realizar consultas parametrizadas internamente.



Sistema de alertas

En este caso es importante entender que una alerta jamás es creada de forma manual por ningún usuario sino que se generan de forma automática en base a los datos recibidos (ya sean simulados o reales).

El flujo de las alertas con datos emulados es:

1. Llega el body de un dato emulado al endpoint `/api/emulatedData/create`.
2. El useCase "CreateEmulatedData" guarda el dato en la DB.
3. A su vez el useCase llama al servicio de dominio "EmulatedImpactAnalysisService" que analiza los 12 sensores de impacto.
4. Si algún sensor supera el umbral establecido (50g), el servicio devuelve la zona y la severidad del impacto.
5. Si hay impacto, se crea una alerta con type: "emulated" y crash apunta al ID del dato emulado.
6. Si no hay impacto devuelve null y no se crea la alerta.

El flujo de las alertas con datos reales es:

1. Llega el body de un dato real al endpoint `/api/realData/create`.
2. El useCase "CreateRealData" guarda el dato en la DB.
3. A su vez el useCase llama al servicio de dominio "RealImpactAnalysisService" que analiza el campo impact.
4. Si supera el umbral (2g) se crea una alerta con type: "real", zone: "front-center" y severity: "moderate" (Hardcodeados porque el sensor real es un único sensor de muy baja potencia enfocado en una zona concreta).
5. Si no hay impacto devuelve null y no se crea la alerta.

Componentes del frontend

Estructura general:

- Pages: Son contenedores simples que montan los componentes. No tienen ningún tipo de lógica y sólo importan y renderizan componentes.
- Components: Poseen la lógica y UI real. Están organizados por página en este caso y no por tipo.
- Layouts: Definen la estructura visual común que siguen un grupo de páginas. En este caso encontramos MainLayout que posee footer y NoFooterLayout que como su nombre indica, no posee footer.
- Router: Define qué páginas se muestran en cada ruta.
- Context: En este caso alberga AuthContext que gestiona el estado de autenticación (Si hay usuario logged o no).
- Services: Archivos que encapsulan las llamadas HTTP al backend usando axios y no el fetch nativo de React.
- Utils: Utilidades compartidas en el frontend como ApiClient.



Páginas principales:

1. LandingPage: Página de inicio pública.
2. Login: Formulario de inicio de sesión.
3. Dashboard: Panel principal de gestión (protegido contra usuarios sin autenticar).
4. Log: Tabla de logs con filtros de búsqueda.
5. AboutUs: Página sobre el equipo y valores.
6. Documentation: Listado de documentos con visor PDF.
7. Demo: Página para solicitar demo del producto.
8. Technology: Tarjetas con la tecnología usada.

Componentes compartidos (Misc):

1. NavBar: Barra de navegación.
2. Footer: Pie de página.
3. LoginForm: Formulario de login.
4. LogsTable: Tabla reutilizable para mostrar datos emulados o reales.
5. ProtectedRoute: Wrapper que redirige al login si no hay sesión.
6. PageTransition: Animación de transición entre páginas.

Filtros y búsqueda

La tabla de logs posee tres filtros; plate (patente), fromDate (desde x fecha) y toDate (hasta x fecha), estos filtros son aplicables tanto para el modo real como para el modo emulado. Los filtros son totalmente combinables entre sí y es importante destacar que el filtro de la patente es parcial. Lo que quiere decir que no es necesario escribir la patente entera.

Funcionamiento:

1. El usuario escribe los inputs en el frontend (patente y fechas).
2. El servicio (RealDataService/EmulatedDataService) envía los filtros como query params al backend.
3. El controller extrae plate, from y to (Alias de fromDate y toDate) de req.query.
4. El useCase lo pasa al repositorio.
5. El repositorio construye la query de Sequelize dinámicamente:
 - Si hay plate → usa Op.like con %plate% sobre la tabla vehicles.
 - Si hay fromDate y toDate → usa Op.between en createdAt.
 - Si solo hay fromDate → usa Op.gte (mayor o igual).
 - Si solo hay toDate → usa Op.lte (menor o igual)
6. Los resultados se devuelven paginados con limit y offset.